Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Asıl ders yaklaşık 00:09:12'de başlar.
- Geçen haftanın tekrarı
  - Mealy ve Moore modeli devreler, state diagram farkları.
- Ardışıl devre tasarımı: dizi tanıyıcı (sequence recognizer)
  - 1101 dizisini tanıyan devrenin state diagram'ı ve durum kodlaması.
- Saklayıcılar (registers)
  - Basit kaydırmalı saklayıcı (shift register).
  - Load girişi eklemenin iki yolu: clock gating ve load-controlled feedback.
  - Shift ve load kontrollü saklayıcı. MUX ile çok fonksiyonlu (sola/sağa kaydırma, paralel yükleme) saklayıcı.
- Saklayıcı transfer işlemleri
  - Datapath ve control unit.
  - Saklayıcı transfer notasyonu, koşullu transfer.
  - Mikro işlemler: transfer, aritmetik, lojik, kaydırma.
  - Toplayıcı-çıkarıcı ile koşullu aritmetik mikro işlemler, C ve V bayrakları.
- Seri toplayıcı (serial adder) örneği
- Sayıcılar (counters)
  - Asenkron (ripple) sayıcı: D ve T flip-flop ile.
  - Senkron ikili sayıcı tasarımı (D flip-flop).
  - Load girişli sayıcıdan BCD (mod 10) ve 9-14 arası sayan sayıcı.
  - Rastgele sayma düzenli sayıcı (0, 1, 3, 2, 4, 6) ve JK geçiş tablosu.

Hocanın Özellikle Vurguladığı Kısımlar

- Mealy ve Moore modellerini ayırt edin.
  - Moore'da çıkış dairenin içine yazılır, okların üzerinde yalnızca giriş bulunur.
- İşlemci içinde clock gating kullanılmaz.
  - Clock yoluna eklenen kapılar gecikme yaratır. Bütün saklayıcı ve sayıcılar clock'un aynı kenarında durum değiştirmelidir.
  - Tercih edilen çözüm load-controlled feedback'tir: devre clock'un önüne değil D girişinin önüne konur.
- "Shift down/up" ifadeleri yerine shift left/right kullanın. Slayttaki "Q0'dan Q3'e" kaydırma shift left'tir.
- Fonksiyon tablosundan flip-flop giriş denklemini yazabilmelisiniz. Çok fonksiyonlu saklayıcıda her flip-flop'un önüne bir MUX konur.
- Ripple (asenkron) sayıcıları bilin, ama tasarımlarda senkron sayıcılar kullanılacak.
- JK geçiş tablosunu ezberlemeyin; karakteristik tablodan kendiniz üretin.
- Senkron sayıcıda kullanılmayan durumlardan (unused states) başlanırsa devrenin nasıl davrandığını kontrol etmek gerekir.
- Load girişli bir ikili sayıcıyı BCD sayıcıya ya da 9-14 arası sayan sayıcıya dönüştürmeyi çalışın.

Kısa Tekrar Notları

- Dizi tanıyıcı (1101), durum kodlaması A = 00, B = 01, C = 11, D = 10:

  - $D_A = AB + BX$, $D_B = X$, $Z = A\bar{B}X$.
- Load-controlled feedback: $D_i = \bar{L}\,Q_i + L\,\text{In}_i$.
- Shift + load saklayıcı (shift left, S = shift, L = load):
  - $D_0 = \bar{S}\bar{L}Q_0 + \bar{S}L\,\text{Data}_0 + S\,\text{SI}$
  - $D_i = \bar{S}\bar{L}Q_i + \bar{S}L\,\text{Data}_i + S\,Q_{i-1}$
- MUX'lu saklayıcı (her bit için bir 4'ten 1'e MUX):
  - 00: Q_i (değişmez), 01: Q_{i-1} (shift left), 10: Q_{i+1} (shift right), 11: Data_i (paralel yükleme).
- Mikro işlem: bir clock periyodunda saklayıcılar üzerinde yapılan en temel işlem.
- Koşullu transfer: $K_1 : R_2 \leftarrow R_1$.
- 4 bitlik seri toplayıcı: iki sayıyı yükleyip toplamak 12 clock sürer.
- Ripple sayıcı: T = 1 olan T flip-flop'lar, her Q bir üst bitin clock'una bağlı.
  - Negatif kenar tetiklemeli: yukarı sayar. Pozitif kenar tetiklemeli: aşağı sayar.
- Senkron ikili sayıcı (D flip-flop):
  - $D_{A_0} = \bar{A}_0$, $D_{A_1} = A_1 \oplus A_0$, $D_{A_2} = A_2 \oplus A_1A_0$, $D_{A_3} = A_3 \oplus A_2A_1A_0$.
- BCD sayıcı: 9 (1001) görülünce load = 1 (Q_3Q_0), yüklenecek değer 0000.
- 9-14 sayıcı: 14 (1110) görülünce load = 1 ($Q_3Q_2Q_1$), yüklenecek değer 1001.
- JK geçiş tablosu: $0 \to 0$: 0X, $0 \to 1$: 1X, $1 \to 0$: X1, $1 \to 1$: X0.

Detaylı Açıklamalar

Dizi tanıyıcı (sequence recognizer)

Devrenin bir X girişi ve bir Z çıkışı vardır. Başta doğrudan (asenkron) reset ile bütün flip-flop'lar sıfırlanır. Devre son dört giriş 1, 1, 0, 1 olduğunda Z = 1 üretir. Diziler örtüşebilir: bir dizinin son 1'i bir sonrakinin ilk 1'i olabilir.

Temsilî örnek (derste yazılan dizi değildir): X = 1101101 için Z = 0001001.

Durumların anlamı:

- A: başlangıç, dizinin hiçbir elemanı yakalanmadı.
- B: bir tane 1 görüldü.
- C: arka arkaya iki 1 (11) görüldü.
- D: 110 görüldü.

Geçişler (X/Z):

Durum	X = 0	X = 1
A	A / 0	B / 0
B	A / 0	C / 0
C	D / 0	C / 0
D	A / 0	B / 1

C'de iken 1 gelirse C'de kalınır: son iki giriş yine 11'dir. D'de iken 1 gelirse dizi tamamlanır, Z = 1 olur ve B'ye gidilir. Çünkü son 1, yeni bir dizinin ilk elemanı olabilir.

Dört durum için iki durum değişkeni (iki flip-flop) yeterlidir. Durumlara kod atamak devrenin karmaşıklığını etkiler. Hoca C = 11, D = 10 atamasının daha basit denklem verdiğini söyledi. A = 00, B = 01, C = 11, D = 10 ile D flip-flop denklemleri:

$$D_A = AB + BX, \qquad D_B = X, \qquad Z = A\bar{B}X$$

Basit kaydırmalı saklayıcı

4 tane D flip-flop arka arkaya bağlanır. Bir flip-flop'un Q çıkışı sağındakinin D girişine gider. Bütün clock girişleri ortaktır. Soldaki flip-flop'un D girişi seri giriş (serial input), en sağdakinin çıkışı seri çıkıştır (serial output). Her clock kenarında veri bir bit kayar.

Load girişi eklemek

Amaç: load = 1 iken paralel girişlerdeki veri saklayıcıya yüklensin, load = 0 iken saklayıcı mevcut değerini korusun.

1. Clock gating. Load bir inverter'dan geçer ve clock ile birlikte bir OR kapısına girer. Load = 1 iken OR'un diğer girişi 0 olur ve clock geçer. Load = 0 iken OR çıkışı sürekli 1 olur ve clock kenarı oluşmaz.

Sorun: inverter ve OR kapısı clock'u geciktirir. İşlemcideki bütün senkron birimler clock'un aynı kenarında durum değiştirmelidir. Gecikmeli clock alan saklayıcı bu eşzamanlılığı bozar. Bir öğrenci inverter'ın kaldırılabileceğini sordu. Hoca, farklı kapılarla da yapılabileceğini ama her durumda en az bir kapının gecikme ekleyeceğini söyledi. Bu çözüm yalnızca bağımsız çalışan devrelerde kullanılabilir.

2. Load-controlled feedback. Clock bütün flip-flop'lara doğrudan bağlanır. Her D girişinin önüne bir seçici devre konur:

$$D_i = \bar{L}\, Q_i + L\, \mathrm{In}_i$$

Load = 0 iken flip-flop kendi çıkışını yeniden yükler (değer korunur). Load = 1 iken yeni veri yüklenir. İşlemcide bu çözüm kullanılır.

Shift ve load kontrollü saklayıcı

Fonksiyon tablosu:

Shift	Load	İşlem
0	0	Değişmez
0	1	Paralel yükleme
1	X	Shift ($Q_0 \to Q_3$ yönünde, yani sola)

Bu tablodan giriş denklemleri yazılır:

$$D_0 = \bar{S}\bar{L}\, Q_0 + \bar{S}L\, \mathrm{Data}_0 + S\, \mathrm{SI}$$

$$D_1 = \bar{S}\bar{L}\, Q_1 + \bar{S}L\, \mathrm{Data}_1 + S\, Q_0$$

Sola kaydırmada Q_0'ın yerine dışarıdan seri giriş (SI), Q_1'in yerine Q_0 gelir.

MUX ile çok fonksiyonlu saklayıcı

İki bitlik mod kontrol girişi ($S_1 S_0$):

$S_1 S_0$	İşlem	D_i
00	Değişmez	Q_i
01	Shift left	Q_{i-1}
10	Shift right	Q_{i+1}
11	Paralel yükleme	Data_i

Denklem AND-OR kapılarıyla da kurulabilir, ama her flip-flop'un önüne bir 4'ten 1'e MUX koymak daha pratiktir. 4 bit için 4 MUX gerekir. Uç bitler için sol ve sağ seri girişler (left/right serial input) kullanılır.

Saklayıcı transfer işlemleri

CPU iki ana bloktan oluşur:

- **Datapath unit:** saklayıcılar, ALU ve kaydırma birimi. Veriyi işler.
- **Control unit:** komutları çözer ve kontrol işaretlerini üretir. Datapath'ten durum bitlerini (V, Z, N, C) alır.

Notasyon (işlemciye özgü değil, genel):

- Saklayıcılar harf ve numarayla gösterilir: R_1, AR (adres saklayıcısı), IR (komut saklayıcısı).
- Parantez saklayıcının bir bölümünü gösterir. Örneğin 16 bitlik bir saklayıcının alt ve üst 8 biti.
- Ok transferi gösterir: $R_1 \leftarrow R_2$.
- Virgülle ayrılan işlemler aynı anda yapılır.
- Köşeli parantez bellek adresini gösterir.
- Koşullu transfer: $K_1 : R_2 \leftarrow R_1$. K_1 aslında R_2'nin load enable ucudur.

Mikro işlem, saklayıcılar veya bellek üzerinde yapılan en temel işlemdir. Bir komut genellikle birkaç clock periyodu sürer. Her periyotta yapılan işlem bir mikro işlemdir. Türleri: transfer, aritmetik (toplama, çıkarma, çarpma, bölme, birler/ikiler tümleyeni, artırma, azaltma), lojik (NOT, AND, OR, XOR) ve kaydırma.

Koşullu aritmetik örnek:

$$\bar{X}K_1 : R_1 \leftarrow R_1 + R_2, \qquad XK_1 : R_1 \leftarrow R_1 + \overline{R_2} + 1$$

Toplayıcı-çıkarıcının kontrol ucu X'tir. K_1, R_1'in load ucudur. En yüksek bitten çıkan elde C bayrağında tutulur. $V = C_n \oplus C_{n-1}$ ile overflow bayrağı üretilir.

Seri toplayıcı

İki 4 bitlik kaydırmalı saklayıcı (A ve B), tek bir tam toplayıcı ve eldeyi tutan bir D flip-flop vardır. Saklayıcıların seri çıkışları toplayıcıya girer. Toplam A'nın seri girişine yazılır. Dışarıdan veri yalnızca B'nin seri girişinden verilebilir.

Soru: saklayıcılar sıfırken iki 4 bitlik sayıyı yükleyip toplamak kaç clock sürer? Cevap 12'dir.

1. İlk 4 clock: birinci sayı B'ye yüklenir. Bu sırada A'ya 0 + 0 yazılır.
2. Sonraki 4 clock: ikinci sayı B'ye yüklenir. Bu sırada B'den çıkan birinci sayı, A'daki sıfırlarla toplanıp A'ya yazılır.
3. Son 4 clock: A ve B toplanır, sonuç A'da oluşur.

Avantaj: tek tam toplayıcı yeterlidir. Dezavantaj: işlem uzun sürer. Bir öğrenci tam toplayıcının clock olmadan nasıl çalıştığını sordu. Hoca, tam toplayıcının kombinasyonel olduğunu söyledi: girişler değiştikçe çıkışı sürekli üretir. Clock yalnızca sonucun flip-flop'lara yüklenmesi için gerekir.

Asenkron (ripple) sayıcılar

Flip-flop'ların clock girişleri ortak değildir. Bir flip-flop'un çıkışı bir üst bitin clock'unu sürer.

D flip-flop ile: her flip-flop'un $\bar{Q}$ çıkışı kendi D girişine bağlanır. Böylece her clock kenarında tümlenir. $\bar{Q}$ ayrıca bir üst bitin clock girişine gider. Pozitif kenar tetiklemeli bu yapı 0, 1, 2, 3 diye yukarı sayar.

T flip-flop ile (4 bit): bütün T girişleri 1'e bağlanır. Q_0 bir sonraki flip-flop'un clock girişine bağlanır, ve böyle devam eder. Flip-flop'lar negatif kenar tetiklemeli ise devre yukarı sayar. Pozitif kenar tetiklemeli olursa aşağı sayar. Hoca bunun zaman diyagramını kendiniz çizmenizi istedi.

JK flip-flop ile de J = K = 1 yapılarak aynı devre kurulabilir.

Senkron ikili sayıcı

Bütün flip-flop'lar ortak clock ile çalışır. 4 bitlik yukarı sayıcı için state table'da her durumun bir fazlası yazılır. 15'ten sonra 0'a dönülür. D flip-flop'ta sonraki durum doğrudan D girişidir. Karnaugh haritalarından:

$$D_{A_0} = \bar{A_0}, \quad D_{A_1} = A_1 \oplus A_0, \quad D_{A_2} = A_2 \oplus A_1 A_0, \quad D_{A_3} = A_3 \oplus A_2 A_1 A_0$$

Kalıp açıktır: her bit, altındaki bütün bitler 1 olduğunda tümlenir. Bu yüzden sayıcı kolayca 5, 6 veya daha fazla bite genişletilir.

Load girişli sayıcıdan başka sayıcılar

Load ve count girişli bir senkron ikili sayıcı kullanılır. Count sürekli 1'dir.

- **BCD (mod 10) sayıcı:** 9 = 1001 görülünce $Q_3 Q_0$ AND kapısından load = 1 üretilir. Paralel girişlere 0000 verilir. Sayıcı 0-9 arası sayar.
- **9-14 arası sayıcı:** 14 = 1110 görülünce $Q_3 Q_2 Q_1$ AND kapısından load = 1 üretilir. 15 hiç oluşmadığı için üç bit yeterlidir. Paralel girişlere 1001 verilir.

Rastgele sayma düzenli sayıcı (JK flip-flop)

Sayma düzeni: 0, 1, 3, 2, 4, 6, 0, … En büyük değer 6 olduğu için 3 durum değişkeni ($A_2 A_1 A_0$) gerekir. 5 ve 7 kullanılmayan durumlardır, sonraki durumları don't care (X) yazılır.

JK geçiş tablosu karakteristik tablodan üretilir:

$Q(t) \to Q(t+1)$	J	K
$0 \to 0$	0	X
$0 \to 1$	1	X
$1 \to 0$	X	1
$1 \to 1$	X	0

Örneğin $0 \to 0$ için iki satır uygundur: "değişmez" (JK = 00) ve "reset" (JK = 01). İkisinde de J = 0, K önemsizdir.

State table ve flip-flop girişleri:

$A_2 A_1 A_0$	Sonraki	$J_2 K_2$	$J_1 K_1$	$J_0 K_0$
000	001	0X	0X	1X
001	011	0X	1X	X0
010	100	1X	X1	0X
011	010	0X	X0	X1
100	110	X0	1X	0X
101	XXX	XX	XX	XX
110	000	X1	X1	0X
111	XXX	XX	XX	XX

Ders bu noktada süre dolduğu için bitti. Sadeleştirme ve kullanılmayan durum analizi bir sonraki derse kaldı (çözüm aşağıda, Ders sonrası notlar'da).

Ders sonrası notlar

- Derste tamamlanamayan JK sayıcının sadeleştirilmiş giriş denklemleri (Karnaugh haritalarından):
  - $J_2 = A_1 \bar{A_0}$, $K_2 = A_1$
  - $J_1 = A_0 + A_2$, $K_1 = \bar{A_0}$

  - $J_0 = \bar{A}_2\bar{A}_1$, $K_0 = A_1$
- Kullanılmayan durumlar bu denklemlerle şöyle ilerler: $5 \to 7 \to 2$. Devre kullanılmayan bir durumda başlasa bile en geç iki clock sonra sayma düzenine girer (kendi kendini düzelten sayıcı). Denklemler ve geçişler Python ile doğrulandı.
- Dizi tanıyıcının denklemleri ($D_A = AB + BX$, $D_B = X$, $Z = A\bar{B}X$) derste yazılmadı. Hocanın önerdiği kodlamayla (A = 00, B = 01, C = 11, D = 10) hesaplanıp bütün geçişler için Python ile kontrol edildi.
- Seri toplayıcının 12 clock sonucu doğrudur. İkinci 4 clock sonunda C flip-flop'unda 0 kalır; çünkü birinci sayı 0 ile toplanırken elde oluşmaz. Bu yüzden son toplama doğru başlar.
- Koşullu aritmetik örnekte "X girişinin komplementi" dendi. X kontrol ucudur. Tümlenen R_2'nin bitleridir.
- Seri toplayıcı anlatılırken birinci sayı "A sayısı" diye anıldı ve sıfırların register B'de olduğu söylendi. Bu yapıda dışarıdan veri B'nin seri girişinden girer. Birinci sayı ikinci 4 clock'ta B'den çıkar ve register A'daki sıfırlarla toplanır.
- Rastgele sayma düzenli sayıcıda "2, 4, 6, 8 kullanılmayan state'ler" dendi. 0, 1, 3, 2, 4, 6 düzeninde kullanılmayan durumlar 5 ve 7'dir.
- Senkron sayıcının denklemlerinin genel hâli: $D_{A_i} = A_i \oplus A_{i-1}A_{i-2}\cdots A_0$.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.